DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CSA06 Design and Analysis of Algorithms
ASSIGNMENT

Assignment 1 - CO1

CO1 Coverage: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Course Outcome: CO1 – Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)
Duration:1week
Scenario — University Student Record Retrieval
A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.
The database may contain:
1,000 → 10,000 → 100,000 → 1,000,000+ records
You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.
Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.
Design Challenge:
The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.
Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Design the Algorithm 5 Marks
Design and propose **at least three meaningfully different algorithmic strategies** for locating a given registration number.
The three approaches must represent **distinct algorithmic strategies**. Iterative and recursive implementations of the **same underlying algorithm will not be considered as two different approaches**.
Your proposed approaches must include:

- At least **one iterative approach**
- At least **one recursive approach**
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation

- Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to **design and select the approaches independently**. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

TASK 2 – Identify the Basic Operation 2 Marks

For each of the proposed algorithms, identify the **basic operation** that predominantly determines its execution time.

Discuss:

- What the basic operation is
- Where it occurs in the algorithm
- Why it is appropriate to use this operation for efficiency analysis
- How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to **relate the selected operation to the behaviour of the algorithm**.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

- The input size (n)
- The number of times the basic operation executes
- How the operation count changes with (n)
- The mathematical expression representing the running time
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the **mathematical reasoning leading to the complexity result**.

Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour 4 Marks

Analyse the performance of each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the **best case**
- What input condition produces the **average case**
- What input condition produces the **worst case**
- How the number of operations changes in each case
- Why these differences occur

Do not simply provide a table containing complexity values. You are expected to **explain the conditions that lead to each case**.

Evidence Expected
A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis **3 Marks**
Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.
Discuss:

- Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant
- How each algorithm behaves as (n) becomes very large
- How the growth rates of the proposed approaches differ
- What the growth behaviour means when the dataset increases from **1,000 to 1,000,000+ records**

You are expected to **interpret the asymptotic behaviour in the context of the given application**, rather than merely state the notation.
Evidence Expected
Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

TASK 6 – Formulate the Recurrence Relation **3 Marks**
For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm.
Clearly identify:

- The input size of the original problem
- The size of the recursive subproblem
- The number of recursive calls
- The work performed outside the recursive call
- The base case
- How these components lead to the recurrence relation

The recurrence must be **derived from your algorithm**, rather than simply writing a known recurrence formula.
Evidence Expected
A correctly formulated recurrence relation with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation **4 Marks**
Solve the recurrence relation obtained in Task 6 using an appropriate analytical method.
You may use:

- Substitution method
- Iteration method
- Master Theorem
- Another appropriate analytical technique

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence

- The resulting complexity
- What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to **show the analytical process**, rather than provide only the final complexity.

Evidence Expected
A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms **4 Marks**
Implement all the proposed algorithmic approaches and experimentally test their behaviour.
Use datasets of approximately:
1,000 → 10,000 → 100,000 → 1,000,000 records
Ensure that:

- All algorithms produce the correct result
- The same or equivalent datasets are used for comparison
- Testing conditions are kept reasonably consistent
- Appropriate test cases are included
- The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is **not merely to demonstrate working code**, but to generate evidence for validating your theoretical analysis.
Evidence Expected
Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

TASK 9 – Record Experimental Performance **3 Marks**
Collect and present experimental performance data for each algorithm across the different input sizes.
Record appropriate measures such as:

- Execution time
- Number of comparisons
- Number of key operations
- Input size
- Other relevant performance measures, where justified

Present the results using appropriate:

- Tables
- Graphs
- Performance plots

Ensure that the presentation allows the different approaches to be easily compared.
Evidence Expected
Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

TASK 10 – Compare Theory with Experimental Behaviour **4 Marks**
Compare your theoretical analysis with your experimental results.

Discuss:

- Whether the observed growth agrees with the theoretical complexity
- Which algorithm scales better as (n) increases
- Any differences between theoretical prediction and actual execution
- Possible reasons for the differences

You are expected to **interpret the results**, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

- Which approach you recommend
- Why it is more appropriate than the alternatives
- Its theoretical efficiency
- Its experimentally observed performance
- Any preprocessing cost involved
- Relevant trade-offs or limitations
- Whether the solution remains practical as the database grows

Your recommendation must be based on **evidence obtained from your own analysis and experimentation**.

Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**.

However, new student records are added only **once per day**.

Re-evaluate your original design in response to these changed operating conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions are affected
- Why the frequency of searches is now important
- Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

- Its expected behaviour under 50,000 searches per hour
- Whether its preprocessing cost is acceptable under the new conditions
- Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

- Describe the revised algorithmic strategy
- Explain what has changed
- Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

- Mathematical complexity analysis
- Experimental results, where appropriate
- Search frequency
- Preprocessing cost
- Dataset size
- Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

MARK DISTRIBUTION

Task	Marks Assigned	Marks Obtained
Task 1 – Algorithm Design	5	
Task 2 – Basic Operation	2	
Task 3 – Mathematical Complexity Analysis	6	
Task 4 – Best/Average/Worst-Case Analysis	4	
Task 5 – Asymptotic Analysis	3	
Task 6 – Recurrence Formulation	3	
Task 7 – Recurrence Solution	4	
Task 8 – Implementation & Testing	4	
Task 9 – Experimental Performance	3	
Task 10 – Theory vs Experiment	4	
Task 11 – Final Design Recommendation	4	
Subtotal	**42**	
Task 12 – Day-4 Design Challenge	**8**	
TOTAL	**50**	